

Aufgabe 1: Schmucknachrichten

Teilnahme-ID: 76626

Bearbeiter/-in dieser Aufgabe:
Quirin Klemm

6. April 2025

Inhaltsverzeichnis

1	Lösungsidee	2
1.1	Perlen besitzen gleichen Durchmesser	2
1.2	Perlen mit unterschiedlichen Durchmesser	4
2	Umsetzung	5
2.1	Erstellung des Baums	5
2.2	Verbesserung des Baums	6
3	Laufzeit und Diskussion	8
4	Beispiele	8
5	Quellcode	8

1 Lösungsidee

Eine Nachricht besteht aus mehreren unterschiedlichen Buchstaben. Für jeden Buchstaben ist ein Codewort zu finden, wobei insgesamt ein präfixfreier Code entstehen muss. Dabei soll die Gesamtlänge der codierten Nachricht möglichst gering sein. Daher wird ein präfixfreier Code gesucht, bei dem häufig vorkommende Buchstaben kurze Codewörter erhalten, während seltenere Buchstaben längere Codewörter zugewiesen bekommen. Im Allgemeinen gilt, solange alle Perlen im Durchmesser gleich sind:

$$c_i = p_i \cdot n_i \quad (1)$$

$$l = \sum_i c_i \quad (2)$$

wobei:

- c_i die Kosten des Buchstaben i ist,
- p_i die Häufigkeit des Buchstaben i in der Nachricht darstellt,
- n_i die Länge des Codewortes für den Buchstaben i ist,
- l die Länge der codierten Nachricht ist.

Wenn sich die Durchmesser der Perlen unterscheiden, stimmt diese Gleichung nicht mehr und muss anders berechnet werden.

Um präfixfreien Code zu erzeugen, wird am besten ein Baum benutzt. Da man durch diesen sicher gehen kann, dass keine Codewort der Präfix eines anderen Codewort ist. Dabei ist jedes Blatt des Baums ein Codewort und jede Kante zu dem Blatt eine Code vom Codewort. Der Baum hat mindestens so viele Blätter wie es verschiedene Buchstaben gibt. Jeder interne Knoten hat dabei maximal so viele Kinder, wie es verschiedene Perlen gibt. Die Ebene des Blattes entspricht die Länge des Codewortes.

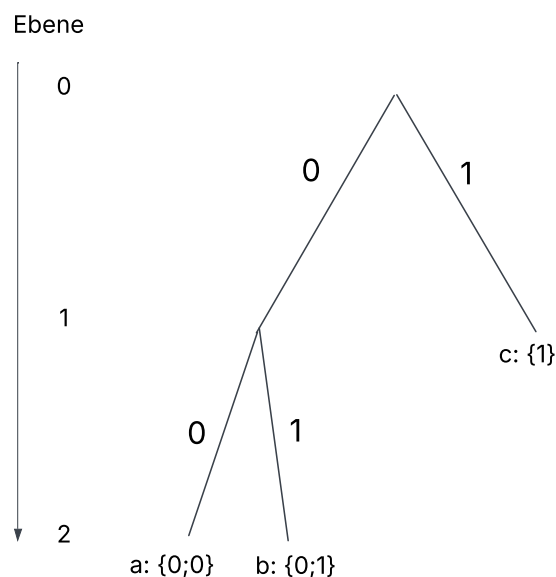


Abbildung 1: simple tree

1.1 Perlen besitzen gleichen Durchmesser

Um möglichst kleinen präfixfreien Code zu erstellen, muss der Buchstaben der am öftesten vorkommt ganz oben eingeordnet werden und die nächsten Buchstaben dann in absteigender Reihenfolge. In Abbildung 1 hätte c die höchste Häufigkeit und dann kommt a und b. Was von a und b die höhere Häufigkeit hat ist egal, da sie bei auf der untersten Ebene sind.

Daher ergibt sich $p_1 \geq p_2 \geq p_3 \dots \geq p_i$ und $n_1 \leq n_2 \leq n_3 \dots \leq n_i$

Zu finden ist also ein Baum, der basierend auf den Häufigkeiten der Buchstaben, seine Blätter so auf seine

Ebene so verteilt, dass die Gleichung (2) möglichst gering ist. Jeder interne Knoten muss dabei mindestens zwei Kinder haben, da ein interner Knoten mit nur einem Kind die Äste des Baums verlängern ohne irgendwelche anderen Vorteile dem Baum zu bringen.

Zu Beginn des Algorithmus wird ein perfekt ausbalancierter Baum erstellt, der so viele Blätter wie Buchstaben enthält (Abbildung 2). Jedes Blatt wird von links nach rechts in aufsteigender Reihenfolge mit einem Buchstaben belegt, und die Gesamtkosten des Baums werden gemäß der Formel (1) berechnet. Dabei ergeben sich deutlich höhere Kosten für die rechten Blätter des Baums im Vergleich zu den linken, da alle Blätter auf derselben Ebene sind, jedoch die Häufigkeit der Buchstaben auf der rechten Seite größer ist. Um das auszugleichen, wird das Blatt mit den höchsten Kosten eine Ebene nach oben verschoben. Dies geschieht indem der Elternknoten des Blattes selber zum Blatt wird und alle Kinder verliert. Dadurch hat der Baum insgesamt zu wenig Blätter, weshalb er erweitert werden muss. Dies geschieht, indem ab der Ebene des entfernten Blattes die Knoten von links nach rechts aufgefüllt werden, bis wieder genügend Blätter vorhanden sind. (Abbildung 3)

Das nach oben Verschieben und unten Neueinfügen der Blätter wird immer wieder wiederholt. (Abbildung 4) Der gesamte Algorithmus ist ein Optimierungsalgorithmus und versucht den Baum beim jeden Durchlauf ein wenig zu verbessern. Das geschieht solange bis die gesamte Kosten des Baums steigen, anstatt zu sinken.-

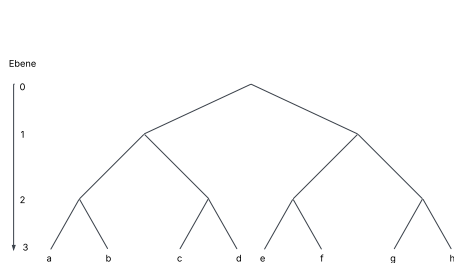


Abbildung 2: Balancierter Baum

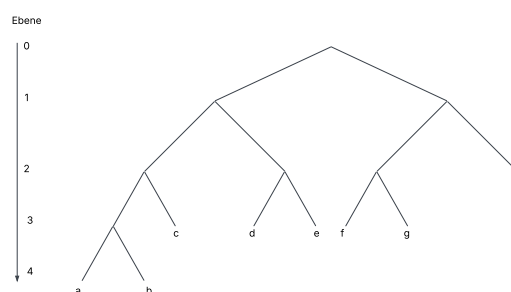


Abbildung 3: Transformation 1

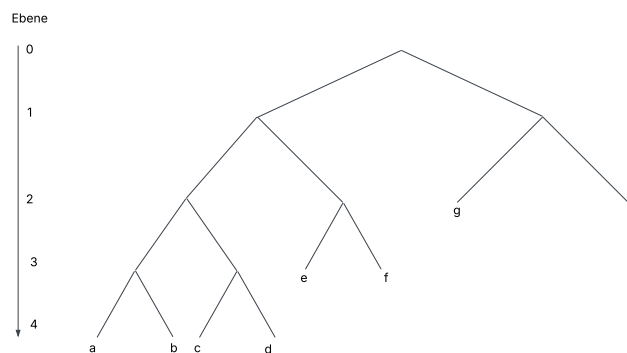


Abbildung 4: Transformation 2

1.2 Perlen mit unterschiedlichen Durchmesser

Das Problem bei unterschiedlichen Durchmesser besteht darin, dass Blätter auf derselben Ebene unterschiedliche Kosten haben können. Daher stimmt die Gleichung (1) nicht mehr, und muss statt dessen berechnet

$$c_i = p_i \cdot (cp_1 + cp_2 + \dots cp_n) \quad (3)$$

wobei:

- cp_n die Kosten/Durchmesser der Perle n ist.

Stellt man nun Bäume nicht nach ihrer Ebene, sondern nach ihren Kosten dar, erhält man einen *lopsided tree*.

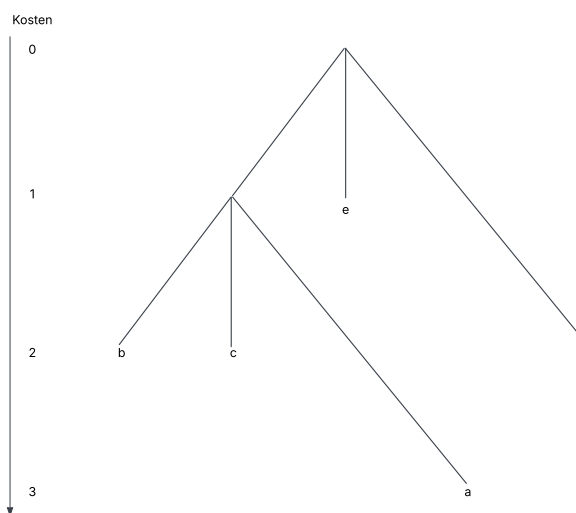


Abbildung 5: lopsided tree: Perlendurchmesser: (1; 1; 2)

Auch hier gilt: Jeder interne Knoten muss mindestens zwei Kinder haben. Bei dieser Art von Bäumen hat jedoch nicht der rechteste Knoten die geringsten Kosten, sondern der oberste. Die Kosten eines Knoten sind:

$$K = K_E + K_S \quad (4)$$

wobei:

- K_E die Kosten seines Eltern Knoten sind
- K_S seine eigenen, neuen Kosten sind

Der Algorithmus bleibt unverändert: Zunächst wird ein Baum erstellt, in dem alle Blätter ungefähr die gleichen Kosten haben. Anschließend wird der Buchstabe mit den höchsten Kosten nach oben verschoben, während der Baum unten erweitert wird. Nach jedem Schritt werden die gesamte Kosten des Baums berechnet (Gleichung 2) und mit den letzten verglichen.

Wenn die Durchmesser alle gleich groß sind, gilt:

$$p_i \cdot (cp_1 + cp_2 + \dots cp_n) = p_i \cdot n_i \quad (5)$$

daher kann der zweite Algorithmus sowohl für gleiche als auch unterschiedliche Durchmesser benutzt werden.

2 Umsetzung

Die Lösung wurde in Python3 geschrieben. Die Eingabetexte werden eingelesen. Die Durchmesser der Perlen werden in einer Liste gespeichert und der Text in einem string. Aus dem Text werden anschließend zwei Listen erstellt: Die erste enthält die Häufigkeit der Buchstaben in absteigender Reihenfolge, die zweite listet die Buchstaben entsprechend dieser Sortierung auf.

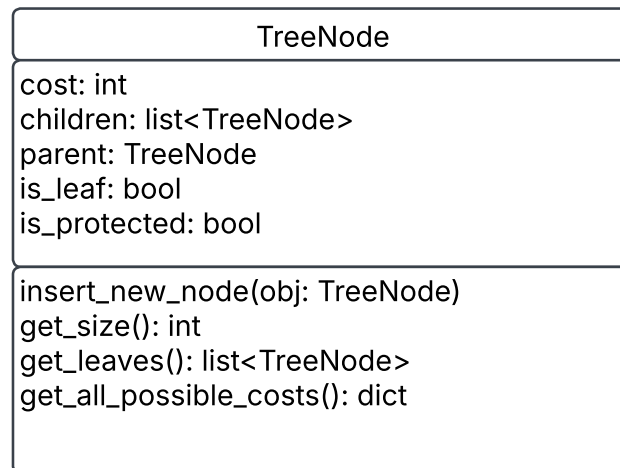


Abbildung 6: TreeNode Klasse

2.1 Erstellung des Baums

Zu Beginn wird der Wurzelknoten, root, mit einem balancierten Baum.

```
1 def create_tree(root: TreeNode) -> TreeNode:
```

Listing 1: Methode create_tree

Dabei werden jeweils alle möglichen neuen Blätter im Baum betrachtet und das Blatt mit den geringsten Kosten wird hinzugefügt. Das passiert solange, bis der Baum genau so viele Blätter hat, wie es Buchstaben gibt. Die möglichen neuen Blätter erhält man über den Aufruf

```
1 root.get_all_possible_costs()
```

```
1 def get_all_possible_costs(self) -> dict:
```

Listing 2: Methode get_all_possible_costs

Die Methode prüft, ob das TreeNode-Objekt noch weitere Kinder aufnehmen kann und nicht protected ist. Falls ja, fügt es sich selbst sowie die Kosten des potenziellen neuen Kindes einem Dictionary hinzu. Anschließend wird die Methode rekursiv für alle bestehenden Kinder aufgerufen, sodass auch deren mögliche Erweiterungen im Dictionary erfasst werden.

Neu Blätter werden über den Aufruf

```
1 root.insert_new_node(obj)
```

hinzugefügt

```
1 def insert_new_node(self, obj: TreeNode):
```

Listing 3: Methode insert_new_node

Die Methode prüft, ob man selbst das `TreeNode` Objekt ist. Falls ja, fügt es einen neuen Kind hinzu. Wenn man vorher noch keine Kinder hatte, werden zwei Kinder hinzugefügt. Ansonsten wird die Methode rekursiv für alle Kinder aufgerufen.

Über den Aufruf

```
1 root.get_size()
```

wird die Anzahl der Blätter ermittelt

```
1 def get_size(self) -> int:
```

Listing 4: Methode `get_size`

Die Methode gibt 1 zurück falls das Objekt ein Blatt ist. Ansonsten wird die Methode rekursiv bei den Kindern aufgerufen und die Summe zurückgegeben

2.2 Verbesserung des Baums

Sobald ein balancierter Baum erstellt wurde, muss dieser noch optimiert werden. Dies geschieht über die Methode

```
1 def find_best_tree(root: TreeNode, last_cost: int) -> Tuple[TreeNode, int]:
```

Listing 5: Methode `find_best_tree`

Die Methode ist rekursiv aufgebaut und erhält bei jedem Aufruf ein `root`-Objekt und die gesamten Kosten aus der vorherigen Iteration. Zuerst versucht sie den Baum zu optimieren, indem sie das Blatt mit den höchsten Kosten eine Ebene nach oben verschiebt und dafür Blätter mit geringen Kosten nach unten verschiebt. Dann berechnet sie die neuen gesamten Kosten des Baums. Sind diese höher als zuvor, übergibt sie ihre Ursprungswerte. Ansonsten ruft sie sich selbst mit den aktualisierten Werten erneut auf.

Das Verbessern des Baums geschieht über die Methode

```
1 def improve_tree(root: TreeNode) -> TreeNode:
```

Listing 6: Methode `improve_tree`

Die Methode holt sich erstmal alle Blätter im Baum über den Aufruf

```
1 root.get_leaves()
```

Diese Liste wird dann aufsteigend sortiert und dann durch

$$c_i \cdot p_i \tag{6}$$

wobei:

- c die Kosten
- p die Häufigkeit
- i der Index des Elements

die Kosten jedes Element berechnet wird. Der Elternknoten des Blatts mit den höchsten Kosten wird daraufhin selbst zu einem Blatt. Dafür werden die Attribute vom Elternknoten

```
1 is_leaf = True
  is_protected = True
3 children = []
```

gesetzt. Um die fehlenden Blätter wieder hinzuzufügen wird noch die Listing 1 aufgerufen.

Die gesamten Kosten des Baums wird durch die Methode

```
1 def calculate_tree(root: TreeNode) -> TreeNode:
```

Listing 7: Methode `calculate_tree`

berechnet. Die Methode holt sich erstmal alle Blätter im Baum über den Aufruf

```
1 root.get_leaves()
```

Die Liste wird dann aufsteigend sortiert und jedes Element wie in der Gleichung 6 berechnet und zusammenaddiert.

3 Laufzeit und Diskussion

4 Beispiele

Genügend Beispiele einbinden! Die Beispiele von der BwInf-Webseite sollten hier diskutiert werden, aber auch eigene Beispiele sind sehr gut – besonders wenn sie Spezialfälle abdecken. Aber bitte nicht 30 Seiten Programmausgabe hier einfügen!

5 Quellcode

Unwichtige Teile des Programms sollen hier nicht abgedruckt werden. Dieser Teil sollte nicht mehr als 2–3 Seiten umfassen, maximal 10.